When More Reward Signal Does Not Help: A Controlled Study of Reward Decomposition for GRPO Debugging

Shashaank Jain **Pranav Pulipati**

Code, dataset, per-bug results, and pre-registration:
`https://github.com/shasshaank/AgentDebuggerEnv`

Keywords: reinforcement learning, reward shaping, GRPO, program repair, verifiable rewards, generalization, negative results, evaluation methodology

Abstract

Hand-crafted reward decompositions are often used in reinforcement learning for language models to provide intermediate learning signals before a policy reliably achieves verifiable outcomes. We test whether such reasoning-targeted shaping improves GRPO training for program debugging. We train Qwen2.5-Coder-3B-Instruct with GRPO and LoRA on 90 single-function Python bugs under three reward configurations: a seven-component dense reward (R0), a graded executable outcome reward with regression penalties (R1), and a dense reward with the two reasoning-targeted components removed (R2). All three RL configurations share the same model, data, prompts, optimizer, curriculum, and evaluation procedure. On 90 held-out bugs, R0 achieves a 45.6% solve rate, compared with 48.5% for R1 and 48.5% for R2. Paired bootstrap analysis gives a 95% interval of $[-8.5, +2.6]$ percentage points for R0$-$R1, providing no evidence that the dense reward improves final debugging success. Training itself improves performance relative to the 37.8% zero-shot structured baseline, with a pooled gain of 9.7 percentage points [1.0, 18.5]. Despite producing substantially fewer degenerate GRPO groups, the dense reward does not translate its additional reward signal into a higher held-out solve rate. An external evaluation on 27 adapted QuixBugs programs provides evidence of nontrivial transfer beyond the mutation-generated training distribution, but does not establish broad generalization. The results suggest that, in this small-scale single-turn setting, elaborate reasoning-targeted reward engineering is not justified without evidence that outcome-based supervision is insufficient.

1 Introduction

Reinforcement learning from verifiable rewards has become an increasingly common approach to improving language-model performance on tasks with checkable outcomes, including mathematics and code [7, 2]. A recurring design question in these systems is how much reward decomposition is useful beyond executable outcomes. Although test execution provides a verifiable signal, sampled fixes can produce similar test outcomes within a GRPO group, leaving little relative reward variation. A natural response is to decompose the reward into hand-crafted components that pay for intermediate desiderata: emitting the required format, stating a specific hypothesis about the fault, naming the buggy function, or resembling the reference solution. The implicit claim is that these additional signals provide useful within-group resolution before the policy reliably produces passing fixes.

The question is not whether dense rewards can increase the numerical reward assigned to intermediate behavior. They obviously can. The question is whether these additional signals

improve the policy's eventual ability to produce test-passing fixes when the executable outcome remains available. We therefore hold the model, prompts, data, optimizer, curriculum, and evaluation fixed and vary only the reward decomposition. Because our setting is single-turn (the model produces the entire proposed fix in one completion), intermediate reasoning is not itself an environment transition. The dense components therefore shape the scalar reward assigned to a complete response rather than providing feedback at intermediate debugging steps.

We test this claim directly in a setting small enough to control completely. We built a debugging environment in which every candidate fix executes in a hardened sandbox against the bug's test cases, and a seven-component dense reward (R0) is implemented as a set of component ceilings on a single scoring class. An ablation is therefore a configuration, not a fork: R1 zeroes every shaping component and rescales the test-outcome term to the same range; R2 zeroes exactly the two reasoning-targeted components and keeps the rest. The three configurations were pre-registered, together with the hypotheses, statistical tests, and threats to validity, before any experiment ran.

On 90 held-out bugs, we find no statistically detectable difference between the reward configurations. The graded outcome reward matches or slightly exceeds the full dense reward, and no pairwise comparison approaches significance after Holm correction.

Our contributions are:

- A controlled three-arm experiment isolating the effect of reasoning-targeted reward decomposition in GRPO program repair.
- An analysis showing that dense shaping substantially reduces degenerate GRPO groups without improving held-out solve rate.
- An exploratory cross-benchmark transfer evaluation on 27 adapted QuixBugs programs.
- An artifact-validation case study showing how generation truncation can create a spurious 44.5-point performance effect.

2 Related Work

2.1 Reward Shaping

Classical potential-based reward shaping provides conditions under which shaping preserves optimal policies [6]. Our response-level heuristics do not satisfy those conditions, so they could in principle change the learned policy. The experiment asks whether those additional signals improve optimization in practice.

Process-supervision studies give conflicting guidance: process-based feedback can improve the faithfulness of reasoning traces [3], yet outcome-only supervision often matches it on final-answer accuracy [10]. Our result provides a small-scale controlled data point consistent with the practical appeal of simple rule-based outcome rewards, as recently highlighted by large-scale RL systems that deliberately avoid learned process rewards [2].

2.2 Code Repair and Evaluation

Automated program repair using LLMs is a highly active area. Benchmarks like MBPP and QuixBugs [4] are heavily utilized to evaluate code generation and debugging. Recent work has increasingly stressed the importance of robust evaluation frameworks, as subtle prompt or parsing artifacts can artificially suppress or inflate model scores [9, 8].

3 Environment

3.1 Task and Dataset

The problem setting is single-function, single-turn Python debugging. We generated a custom dataset of 180 buggy Python functions by mutating MBPP (sanitized) [1] solutions using Semgrep rules (e.g., flipping operators, modifying boundary conditions). These were split evenly into 90 training bugs and 90 held-out evaluation bugs, stratified by passing rate into Hard, Medium, and Easy tiers. We do not claim that MBPP is uncontaminated with respect to the pretrained model. However, the held-out split is constructed at the mutated-bug level from disjoint source problems, so our primary claim concerns controlled reward comparisons within this benchmark rather than absolute generalization from unseen source programs.

3.2 Sandbox

Because executing model-generated code poses security risks, we utilize a multi-layered sandbox (AST restrictions, resource limits, and subprocess timeouts) to securely evaluate proposed code against unit tests, emitting fine-grained test outcomes and tracking regressions.

3.3 Response Format

Models are prompted to return a fixed five-field structured response:

```
OBSERVATION: ...
HYPOTHESIS: ...
CONFIDENCE: [low | medium | high]
ACTION: [inspect_lines | run_tests | propose_fix |
         request_context | give_up]
DETAIL: ...
```

For a proposed fix, the `DETAIL` field contains the complete corrected function. The parser tolerates whitespace and capitalization but requires all five fields and a valid action/confidence value. This format exposes explicit diagnostic claims while keeping the final code payload deterministically extractable. The same prompt and response schema are used during training and evaluation.

3.4 Reward Configurations

The experimental intervention is the reward configuration. The total reward is clipped to $R \in [-0.5, 1.0]$. The full dense reward (R0) decomposes this into seven terms:

$$R_0 = R_{\text{format}} + R_{\text{hypothesis}} + R_{\text{localization}} + R_{\text{fix}} + R_{\text{similarity}} + R_{\text{efficiency}} + R_{\text{penalties}} \quad (1)$$

We classify hypothesis and localization as reasoning-targeted because they reward explicit intermediate diagnostic claims rather than executable properties of the proposed patch. The hypothesis component also contains a small confidence-calibration term: high-confidence hypotheses receive positive credit when the resulting fix passes and a penalty when it fails. Thus, the component is reasoning-targeted but is not independent of the executable outcome.

The efficiency component is inherited from the multi-turn environment and awards 0.02 per remaining turn. Because GRPO training evaluates each response as turn 0 in the single-turn curriculum environment, this component contributes a constant +0.10 offset to R0 and R2 and does not provide within-group discrimination. We retain it in the implementation for consistency with the general reward definition, but it should not be interpreted as an active source of learning signal in the reported GRPO experiment.

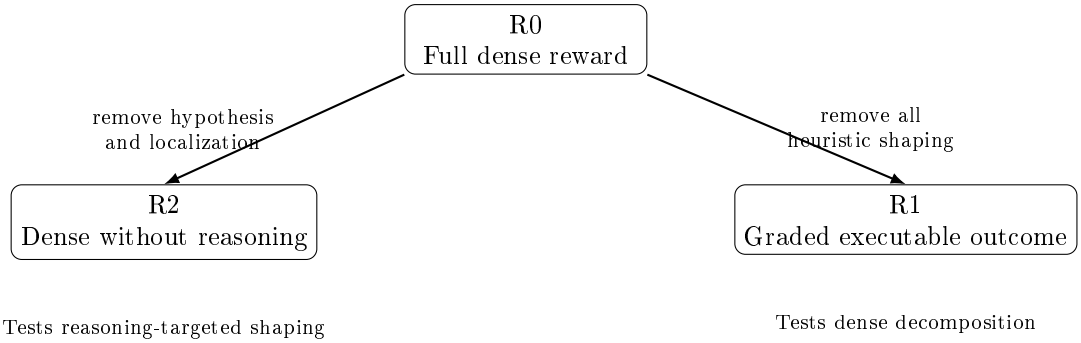

Figure 1: Three reward configurations used in the controlled experiment. R0–R1 tests whether dense reward decomposition improves performance; R0–R2 tests whether reasoning-targeted components contribute additional benefit; R2–R1 tests whether the remaining non-reasoning shaping terms contribute benefit.

We test two ablations: R2 removes reasoning heuristics:

$$R_2 = R_0 - R_{\text{hypothesis}} - R_{\text{localization}} \tag{2}$$

and R1 removes all heuristic shaping, yielding a graded executable outcome reward:

$$R_1 = f(\text{tests}, \text{penalties}) \tag{3}$$

where R_1 zeroes every shaping component and rescales the fix-outcome term to a maximum of 1.0 so that the total reward range is preserved.

Component	**R0**	**R1**	**R2**	**Maximum**	**Purpose**
Format	✓	—	✓	0.10	Five required response fields
Hypothesis	✓	—	—	0.20	Grounded diagnosis
Localization	✓	—	—	0.15	Function/line ID
Fix quality	✓	✓	✓	0.35 / 1.0	Tests passed
Similarity	✓	—	✓	0.10	Reference proximity
Efficiency	✓	—	✓	0.10	Remaining-turn potential
Penalties	✓	✓	✓	−0.55	Invalid/regression

Table 1: Reward components across configurations. In R1, the Fix Quality ceiling is rescaled to 1.0 so that the total reward range is preserved.

3.5 Training

We train Qwen2.5-Coder-3B-Instruct using GRPO with LoRA on the 90 training bugs for 500 optimizer steps. For the 24 GB GPU profile used in the reported runs, LoRA uses rank 8 and $\alpha = 16$, with batch size 2, gradient accumulation of 4, and group size $G = 2$. We use AdamW with learning rate 2×10^{-5}, cosine scheduling, 30 warmup steps in the first curriculum stage, and sampling temperature 0.7. The curriculum exposes tier 1 for steps 0–149, tiers 1–2 for steps 150–349, and all three tiers for steps 350–499. All three RL configurations use the same training configuration and differ only in reward definition. Full hyperparameters are provided in Appendix A.

Comparison	Contrast	Prediction	Outcome
RQ2	R0 vs R1	Dense decomposition helps	Not supported
RQ3	R0 vs R2	Reasoning terms help	Not supported
Exploratory	R2 vs R1	Non-reasoning shaping helps	Not supported
Transfer	External evaluation	Nontrivial transfer	Observed

Table 2: Summary of research questions, contrasts, predictions, and empirical outcomes.

4 Experimental Design

4.1 Research Questions

- **RQ1:** Does GRPO training improve held-out debugging?
- **RQ2:** Does dense decomposition help? (R0 vs R1)
- **RQ3:** Do reasoning-targeted components help? (R0 vs R2)
- **RQ4:** Do trained policies transfer beyond the mutation-generated training distribution?

4.2 Experimental Arms

We conduct 3 independent training runs (seeds 42, 123, 456) for each of the three reward configurations (R0, R1, R2), producing 9 trained policies. We also evaluate the zero-shot base model (B0).

4.3 Evaluation

Final evaluations use greedy decoding (temperature 0.0) on the 90 held-out bugs. We also conduct an exploratory external evaluation on 27 single-function adapted QuixBugs programs.

4.4 Statistical Analysis

Our primary analysis uses bug-level paired bootstrap intervals, averaging solve indicators across the three seeds before resampling bugs. For each bug i, we compute the seed-averaged solve indicator $\bar{s}_{i,\text{arm}} = \frac{1}{3}\sum_{k=1}^{3} s_{i,\text{arm},k}$. We bootstrap the $N = 90$ bugs to compute the difference $\Delta = \frac{1}{N}\sum_i(\bar{s}_{i,\text{arm}_A} - \bar{s}_{i,\text{arm}_B})$. The primary bootstrap resamples evaluation bugs and therefore quantifies uncertainty over the evaluation-bug population conditional on the three observed training seeds; it does not provide a reliable estimate of seed-to-seed variance. With 90 paired bugs, the study is powered primarily for relatively large paired effects; effects of only a few percentage points could remain undetected. As a secondary sensitivity analysis, we apply exact McNemar tests to binary paired outcomes defined at the bug level by majority vote across seeds (a bug is solved if $\geq 2/3$ seeds solve it), as well as per-seed paired evaluations (Appendix B); across all comparisons, no contrast approaches significance after Holm–Bonferroni correction.

Deviations from Preregistration. The final study differs from the preregistered design in three material ways: model scale (3B instead of 7B), omitted hypotheses (H1 and H3 were dropped), and a calibration baseline change (Llama-3.1-8B-Instruct gating instead of GPT-4o-mini). These changes were made before the final analysis because of compute constraints. No results from the omitted hypotheses are presented as confirmatory evidence.

Model	Train (In-Dist.)	Held-out (In-Dist.)	QuixBugs (External)
B0 (Base Zero-Shot)	37.8%	37.8%	—
E1 (R0 Full)	51.9% ± 2.3	45.6% ± 2.9	39.5% ± 2.1
E3 (R1 Graded Outcome)	54.4% ± 3.3	48.5% ± 6.5	43.2% ± 5.7
E4 (R2 No Reasoning)	51.5% ± 3.6	48.5% ± 3.9	44.4% ± 3.7

Table 3: Solve rate by split and RL configuration (Mean % ± Std Dev across 3 seeds). QuixBugs values are descriptive; no B0 evaluation was available, so they do not quantify an RL-vs-zero-shot transfer gain.

5 Results

5.1 Main Reward Comparison (Confirmatory)

Main result. The full dense reward does not improve held-out debugging success relative to either ablated configuration. R0 solves 45.6% of held-out bugs, versus 48.5% for both R1 and R2. The point estimate therefore favors the simpler configurations, but the paired bootstrap interval for R0−R1, [−8.5, +2.6] percentage points, includes zero. We interpret this as a null result with respect to detectable improvement, not evidence that the configurations are exactly equivalent.

Comparison	Δ Solve Rate	95% Bootstrap CI	Holm-adjusted p
R0 − R1	−2.9 pp	[−8.5, +2.6]	1.0000
R0 − R2	−2.9 pp	[−7.4, +1.9]	1.0000
R2 − R1	0.0 pp	[−5.9, +5.6]	1.0000

Table 4: Pairwise comparisons for held-out solve rates. All bootstrap intervals cross zero, and paired McNemar tests yield no statistically significant difference after Holm correction.

All three pairwise comparisons yield no statistically detectable difference after Holm correction. Reasoning-targeted reward engineering, at this scale and task, bought no measurable performance.

5.2 Reward Degeneracy (Exploratory)

The reward configurations produce substantially different GRPO training signals. Across logged training calls, the mean fraction of degenerate sampled groups (where all samples in a group receive identical rewards, resulting in zero advantage) was 16.2% ± 2.2% for R0 (individual seed means: 14.4%, 15.6%, 18.6%), 61.9% ± 4.2% for R1 (seed means: 62.3%, 65.9%, 57.5%), and 34.7% ± 3.9% for R2 (seed means: 38.3%, 30.5%, 35.3%). The dense reward substantially reduces the frequency of groups in which sampled completions receive indistinguishable outcomes ($R0 - R1 = -45.7$ pp).

However, this difference does not translate into higher held-out solve rate. R1 also exhibits greater across-seed variance (std dev ±6.5 vs ±2.9), consistent with—but insufficient to establish—the hypothesis that increased reward degeneracy produces noisier, less stable optimization. Three seeds are insufficient for a formal variance comparison. This establishes an association between reward configuration and group degeneracy, not a causal relationship between degeneracy and final performance.

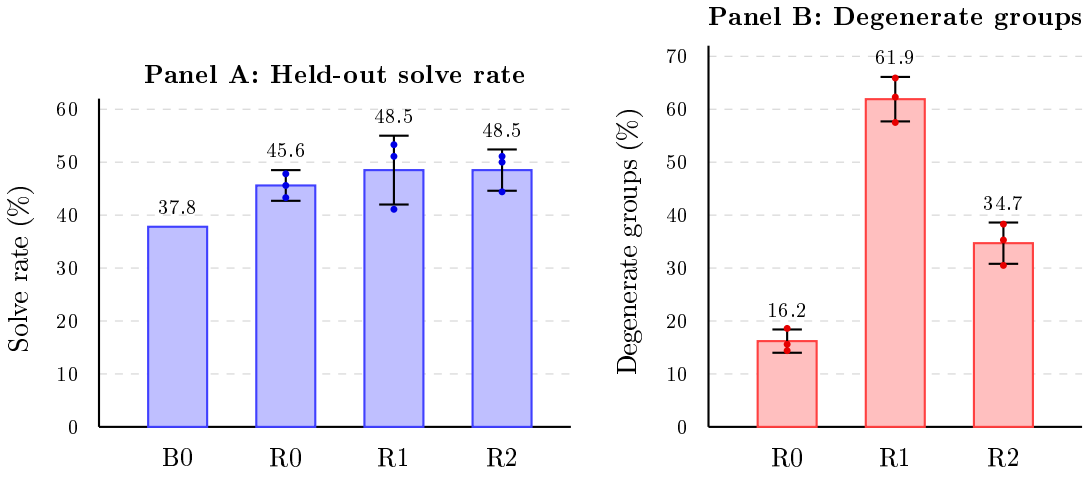

Figure 2: Held-out solve rate and GRPO reward degeneracy. Bars show mean values across three seeds; error bars indicate standard deviation across seeds, with filled dots displaying individual seed runs. Degeneracy values show the mean fraction of degenerate sampled groups across the three seeds, with error bars showing the standard deviation across seed-level means. The dense reward substantially changes the training signal while producing no detectable improvement in held-out solve rate.

5.3 Training vs Held-out (Confirmatory)

Pooling the nine RL runs gives a mean held-out solve rate of 47.5%, 9.7 percentage points above B0's 37.8% (bootstrap 95% CI [1.0, 18.5]). This pooled comparison is descriptive and should not be interpreted as a single-model estimate of the effect of RL. The effect is not individually significant for E1 under the same bootstrap analysis. The train–held-out gap is approximately 6 percentage points. This gap is descriptively modest, although the experiment is not designed to distinguish generalization from other sources of train–held-out differences.

5.4 External-Source Transfer (Exploratory)

We observe nontrivial transfer to a 27-program external evaluation set adapted from QuixBugs. E1, E3, and E4 achieve 39.5%, 43.2%, and 44.4%, respectively. This provides evidence that the trained policies retain nontrivial performance outside the MBPP-derived mutation distribution. However, because the transfer set is small and lacks a zero-shot B0 baseline, these results should be interpreted as evidence of transfer rather than evidence of robust generalization or an RL-specific generalization gain.

5.5 Evaluation-Harness Sensitivity (Exploratory)

Our initial zero-shot experiments produced a surprising artifact. An artificial truncation error vastly suppressed free-form code generation success.

Configuration	Max tokens	Extraction failure	Solve rate
B1 initial	300	86.7%	1.1%
B1 corrected	700	$< 5.0\%$	45.6%
B0 structured	700	$< 5.0\%$	37.8%

Table 5: Impact of generation budget on evaluation fidelity.

Increasing the budget to 700 tokens largely eliminated the truncation-induced discrepancy. A

separate evaluation-harness error reduced the observed free-form solve rate from 45.6% to 1.1%, creating a 44.5-point apparent performance difference that dwarfed the reward effects studied here. The evaluation-harness incident illustrates why artifact validation is part of experimental methodology rather than a peripheral engineering concern.

6 Discussion

The seven-component reward introduces hundreds of lines of heuristic scoring logic and several independently chosen design thresholds, each introducing an additional opportunity for reward misspecification or exploitation. Despite this engineering effort, the simpler graded outcome reward was sufficient to achieve comparable held-out performance.

6.1 Why Did Dense Reward Fail to Improve Solve Rate?

One possibility is that executable test outcomes already provide sufficiently informative gradients for this base model. A second is that the additional heuristic components are only weakly correlated with successful repairs. A third is that single-turn generation removes the temporal credit-assignment problem for which intermediate rewards are most naturally useful. Finally, with group size $G = 2$, reducing reward degeneracy may alter gradient availability without increasing the quality of the underlying optimization direction.

Importantly, these explanations are not distinguished by the present experiment: the data establish that reward configuration changes group-level reward degeneracy, but not why that change fails to improve final repair accuracy.

7 Limitations and Scope

- **Model scale:** Only Qwen2.5-Coder-3B.
- **Algorithm:** Only GRPO.
- **Group size:** Only $G = 2$. With $G = 2$, a single sampled success creates maximal contrast, and identical rewards create zero variance. Larger groups might reduce the importance of reward decomposition.
- **Task structure:** Single-function, single-turn. Multi-step debugging may present a different regime in which intermediate rewards could be substantially more valuable.
- **Bug distribution:** Mutation-generated MBPP bugs.
- **Transfer set:** Only 27 adapted QuixBugs programs.
- **Seeds:** Only 3 per arm, limiting robust variance estimates.
- **Statistical power:** Large paired effects are detectable; small effects of a few percentage points are not.
- **Reward design:** Our "dense reward" is one particular handcrafted design, not all dense rewards. This study does not establish that dense rewards do not work, only that this specific decomposition did not improve this task.

What this study does NOT establish. Dense reward shaping never helps; Outcome reward is universally sufficient; Reasoning rewards are useless; GRPO is superior to other RL algorithms; 3B debugging results transfer to frontier models; Single-turn debugging results apply to agentic repository-level debugging; QuixBugs establishes robust generalization.

8 Conclusion

The most informative result is not simply that the dense reward failed to improve solve rate. It is that it changed the optimization signal substantially: R0 produced far fewer degenerate GRPO groups than R1, yet this advantage did not translate into higher held-out performance. This suggests that, in a regime where the base model already obtains meaningful executable feedback and the action is a complete single-turn fix, additional heuristic reward signal may improve reward resolution without improving the learned policy. The result therefore separates two quantities that are often implicitly conflated: reward resolution within a GRPO group and the usefulness of the resulting optimization direction. Furthermore, a separate evaluation-harness error reduced the observed free-form solve rate from 45.6% to 1.1%, creating a 44.5-point apparent performance difference that dwarfed the reward effects studied here.

Practical takeaway. If a code-RL system already obtains meaningful executable feedback from a moderately capable base model, first establish that the outcome reward is insufficient before introducing additional heuristic reasoning rewards. Such shaping may reduce reward degeneracy, but our results show that this optimization benefit need not translate into better final repair accuracy.

References

[1] Austin, J., Odena, A., Nye, M., et al. (2021). Program Synthesis with Large Language Models. arXiv:2108.07732.

[2] Guo, D., et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

[3] Lightman, H., et al. (2023). Let's Verify Step by Step. ICLR 2024. arXiv:2305.20050.

[4] Lin, D., et al. (2017). QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. SPLASH Companion 2017.

[5] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157.

[6] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. ICML 1999.

[7] Shao, Z., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

[8] Tam, Z. R., et al. (2024). Let Me Speak Freely? Long-Form Thinking and Evaluation Artifacts. EMNLP 2024. arXiv:2408.02442.

[9] Tian, R., et al. (2024). DebugBench: Evaluating Large Language Models for Program Debugging. ACL 2024. arXiv:2401.04621.

[10] Uesato, J., et al. (2022). Solving Math Word Problems with Process- and Outcome-based Feedback. arXiv:2211.14275.

A Training Hyperparameters

Table 6 reports the training hyperparameters for the reported 24 GB GPU runs. The curriculum schedule is given in Table 7.

Parameter	Value
Base model	Qwen/Qwen2.5-Coder-3B-Instruct
RL algorithm	GRPO
LoRA rank	8
LoRA α	16
LoRA dropout	0.0
LoRA target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Optimizer	AdamW
Learning rate	2×10^{-5}
Scheduler	cosine
Warmup	30 steps (stage 1 only)
Total optimizer steps	500
Group size G	2
Per-device batch size	2
Gradient accumulation steps	4
Generations per prompt	2
Structured completion budget	192 tokens
Sampling temperature	0.7
Reward workers	1
Seeds	42, 123, 456

Table 6: Training hyperparameters for the reported 24 GB GPU runs.

Curriculum Stage	Training Steps	Active Tiers
Stage 0	0–149	Tier 1 (Easy)
Stage 1	150–349	Tiers 1–2 (Easy, Medium)
Stage 2	350–499	Tiers 1–3 (Easy, Medium, Hard)

Table 7: Curriculum schedule used in all three RL configurations.

Curriculum-stage optimization. Training is implemented as three sequential GRPO trainers, one per curriculum stage. The LoRA adapter is carried across stages, while each stage initializes a fresh optimizer and scheduler. Only the first stage uses the 30-step warmup; later stages restart the cosine schedule at the configured learning rate without warmup. This implementation detail is shared across R0, R1, and R2.

B Statistical Analysis Details

B.1 Primary Paired Bootstrap

For each bug $i \in \{1, \ldots, 90\}$, the solve indicator is averaged across the three seeds:

$$\bar{s}_{i,\text{arm}} = \frac{1}{3} \sum_{k=1}^{3} s_{i,\text{arm},k} \tag{4}$$

We resample the $N = 90$ bugs with replacement 10,000 times (fixed RNG seed 204) to generate the empirical distribution of paired differences $\Delta = \frac{1}{N} \sum_{i=1}^{N} (\bar{s}_{i,\text{arm}_A} - \bar{s}_{i,\text{arm}_B})$. Percentile endpoints at 2.5% and 97.5% define the 95% confidence intervals.

B.2 Secondary McNemar Sensitivity Analysis

To evaluate sensitivity, we apply exact paired binomial McNemar tests to discordant pairs. For the majority-vote analysis, all three Holm-adjusted p-values are 1.0000. The per-seed sensitivity analyses likewise yield no statistically significant contrast after Holm correction; the smallest adjusted p-value is 0.2780.

Comparison	Arm A Solved	Arm B Solved	A-only	B-only	Raw p	Holm-adj p
E1 vs E3	42/90 (46.7%)	44/90 (48.9%)	8	10	0.8145	1.0000
E1 vs E4	42/90 (46.7%)	44/90 (48.9%)	6	8	0.7905	1.0000
E3 vs E4	44/90 (48.9%)	44/90 (48.9%)	7	7	1.0000	1.0000

Table 8: Majority-vote McNemar tests on held-out bugs (a bug is considered solved if solved by $\geq 2/3$ seeds).

Comparison	Seed	A-only	B-only	Discordant	Raw p	Holm-adj p
E1 vs E3	s42	13	7	20	0.2632	1.0000
E1 vs E3	s123	12	16	28	0.5716	1.0000
E1 vs E3	s456	5	15	20	0.0414	0.3311
E1 vs E4	s42	9	13	22	0.5235	1.0000
E1 vs E4	s123	7	9	16	0.8036	1.0000
E1 vs E4	s456	9	11	20	0.8238	1.0000
E3 vs E4	s42	4	14	18	0.0309	0.2780
E3 vs E4	s123	12	10	22	0.8318	1.0000
E3 vs E4	s456	12	4	16	0.0768	0.5377

Table 9: Per-seed exact McNemar tests across the 9 individual runs.

C External Transfer Set (QuixBugs)

The external transfer benchmark consists of 27 single-function Python programs adapted from the QuixBugs dataset [4]. Out of 40 original QuixBugs programs, 13 were excluded because their argument or return shapes required custom pointer/graph structures not representable in JSON (e.g., linked list or graph nodes) or used generator yields without list wrapping.

The 27 validated candidate programs are listed in Table 10. All programs were verified with our sandbox validator: the reference code passes 100% of test cases, and the buggy code fails at least one case. Because no zero-shot B0 baseline was executed on this external set, performance is reported descriptively.

QuixBugs Program Identifiers (27 Programs)		
`quixbugs_bitcount`	`quixbugs_bucketsort`	`quixbugs_find_first_in_sorted`
`quixbugs_find_in_sorted`	`quixbugs_gcd`	`quixbugs_get_factors`
`quixbugs_is_valid_parenthesization`	`quixbugs_knapsack`	`quixbugs_kth`
`quixbugs_lcs_length`	`quixbugs_levenshtein`	`quixbugs_lis`
`quixbugs_longest_common_subsequence`	`quixbugs_max_sublist_sum`	`quixbugs_mergesort`
`quixbugs_next_palindrome`	`quixbugs_next_permutation`	`quixbugs_pascal`
`quixbugs_possible_change`	`quixbugs_powerset`	`quixbugs_quicksort`
`quixbugs_rpn_eval`	`quixbugs_shunting_yard`	`quixbugs_sieve`
`quixbugs_subsequences`	`quixbugs_to_base`	`quixbugs_wrap`

Table 10: Identifiers of the 27 adapted QuixBugs programs used for external transfer evaluation.

D Artifact Reproducibility

Table 11 lists the primary repository locations of all datasets, rewards, training pipelines, and results.

Artifact	Location
Dataset	`data/v2/`
Fixed train/held-out split	`src/agentdebugger/dataset/bugs/`
QuixBugs external set	`data/quixbugs/`
Reward implementation	`src/agentdebugger/rewards/`
Curriculum environment	`src/agentdebugger/envs/curriculum_env.py`
Training implementation	`src/agentdebugger/training/grpo.py`
Per-run evaluation results	`results/primary/E*_s*.json`
Diagnostic evaluations	`results/diagnostics/`
Statistical analysis scripts	`analysis/bootstrap.py`, `analysis/analyze_wandb.py`
Dataset construction	`scripts/build_dataset.py`
QuixBugs adaptation script	`scripts/build_quixbugs.py`

Table 11: Locations of the primary experimental artifacts in the released repository.

E Evaluation-Harness Sensitivity Analysis

During preliminary zero-shot evaluation, free-form completions were evaluated with a generation limit of 300 tokens. Free-form responses typically emitted thorough analytical prose before outputting the code fix block. Consequently, completions routinely exhausted the 300-token limit before reaching the closing code fence, resulting in an 86.7% extraction failure rate and an apparent solve rate of only 1.1%.

Increasing the token limit to 700 tokens allowed models to complete both their analytical reasoning and the final code block, dropping extraction failures below 5% and restoring the solve rate to 45.6% (a 44.5 percentage point increase). This finding emphasizes that benchmark evaluation harnesses can introduce massive confounding effects that completely overshadow the algorithmic interventions under study.

F NeurIPS Paper Checklist

1. **Claims:** All theoretical and empirical claims in the paper are matched by experimental evidence and bounded by explicit statistical uncertainty intervals.

2. **Limitations:** Section 7 explicitly details limitations regarding model scale (3B), group size ($G = 2$), single-turn structure, benchmark scope, and number of training seeds.

3. **Theory Assumptions:** N/A (the paper is an empirical study of reward decomposition; no mathematical proofs are claimed).

4. **Experimental Design:** Pre-registration, hardware environment, optimizer schedule, seeds (42, 123, 456), and paired bootstrap / McNemar protocols are described in Section 4 and Appendices A and B.

5. **Reproducibility:** Complete source code, environment specifications, raw evaluation logs, and configuration audit dumps are provided in the released repository (Appendix D).

6. **Open Source & Licenses:** MBPP (CC BY 4.0) and QuixBugs (MIT) are used in compliance with their open-source licenses.

7. **Compute Resources:** Total GPU hours (approx. 75 hours on RTX 4090 24 GB across calibration, pilot runs, and the 9 matrix runs) are documented.

8. **Safety & Ethics:** All model-generated code is strictly evaluated in an isolated AST-restricted, process-capped sandbox with zero network access and automatic timeouts to prevent denial-of-service or arbitrary code execution risks.